Lev Poliakov

Senior Game Developer / Gameplay Programmer · Unreal Engine 5 (UE5) · C++

progsag@gmail.com | github.com/sungrow | progsag.com | linkedin.com/in/sungrow

Da Nang, Vietnam · Remote, 08:00-16:00 CET · Open to relocation (EU) | 6+ Years Experience | English C1

PROFESSIONAL SUMMARY

Gameplay programmer. 6+ years of C++ in Unreal Engine, UE4 through UE 5.8. One title shipped on Steam. Two sports simulation titles in development now. Main areas: gameplay systems, game AI (StateTree), cameras and in-game cinematics (Sequencer), and the tooling around them. Performance work as well: on the shipped title, 4 FPS to playable and save/load times cut by 90%. I own systems end to end, from the design doc to the editor tools around them, on a team of 40+.

TECHNICAL SKILLS

Programming Languages:	C++ (C++14/17/20), C, TypeScript, Python, Go, Bash, Blueprints
Unreal Engine 4 / 5:	UE4, UE5 (incl. 5.6 and 5.8). Gameplay Ability System (GAS), StateTree, Behavior Trees, Sequencer (Level Sequence, MovieScene, custom tracks), CommonUI, UMG, Motion Matching (PoseSearch), MotionWarping, Animation Blueprints, AnimDynamics, RigidBody anim nodes, MetaHuman, Chooser, ControlRig, Niagara, MovieRenderPipeline
Game AI:	AI character behavior on StateTree, avoidance with path prediction, animation blending and retargeting
Editor Tooling & Tests:	plugin and editor module development, Sequencer track editors, asset validators, commandlets, Editor Utility Widgets, Unreal Automation tests
Graphics & Performance:	DirectX, OpenGL, Vulkan, HLSL, RenderDoc, profiling, multithreading, concurrency, cache optimization, memory management
Tools & Infrastructure:	Git, CMake, JetBrains Rider, Visual Studio, Jira, AWS, S3, Qt6, Emscripten
Practices:	Agile, Scrum, technical design documents, code review, CI/CD (PowerShell build pipelines), AI-assisted development (Claude Code)

PROFESSIONAL EXPERIENCE

Senior Software Engineer (Gameplay, Unreal Engine), 4People (Remote)

Mar 2025 – Present

Projects: two sports simulation titles with broadcast-style presentation. C++, team of 40+. On the volleyball title: top five of 40+ contributors by commit count, ~550 commits in 20 months.

Tennis title, UE 5.8

Aug 2026 – Present

- In-game cinematics system, built by me: three plugins plus game code, about 10,000 lines of C++ in the first delivery. Server commands start master Level Sequences through StateTree tasks; actors are assigned to sequence roles at run time.
- Custom Sequencer signal track with its own track editor: keys on the timeline reach gameplay code as tagged events. Takes are picked without repeats; a sequence can exit early at a marked frame on a server response.
- Rigid-body anim node that detects a teleport at a camera cut. Hair physics keeps its pose instead of snapping.
- Gameplay camera plugin with handoff between cinematics and gameplay. Asset validators, automation tests, docs for programmers and animators.

Volleyball title (6v6), UE 5.6

Mar 2025 – Present

- Camera operator plugin, built by me. Rule-based: camera behavior composed from And/Or/Not condition nodes, fired by triggers. Ball tracking without gimbal lock. The rest of the camera system runs on it.
- Rally replay and serialization system. Records and replays full rallies. Captures RNG for deterministic playback; regenerates ball trajectories from stored variation data.
- AI characters around the court: coaches, line referees, extras. StateTree behavior in C++, animation blending and retargeting, avoidance based on the player's predicted path. Rewrote the old trainer subsystem into the current coach architecture.

- Match commentary (shared ownership): phrase conditions on rally events. Server-driven match flow: serve, end-of-rally reactions, side changes, timeouts.
- Shipping-build plugin that keeps saves and graphics settings in the build folder. Shipping-only crash fixes, debug overlays, editor utilities.

Technologies: C++, Unreal Engine 5.6 / 5.8, StateTree, Sequencer, Animation Blueprints, AnimDynamics, MetaHuman, Git, Jira

Contract Software Engineer (C++, Unreal Engine), Self-employed (Remote)

Oct 2024 – Feb 2025

- **Clapperheads, on the game Zoochosis (UE5, Jan – Feb 2025):** reworked the inventory system and the animal and monster behavior; fixed critical gameplay issues before a content update.
- **2040World (Qt6, C++, Oct 2024 – Jan 2025):** cleaned up and optimized system components, active bug count down ~70%; reworked the UI library against updated design mockups; tightened backend, launcher and game-module integration.

Technologies: C++, Unreal Engine 5, Blueprints, Qt6, CMake, AWS, S3

Software Engineer (C++), Balancy (Remote)

Mar 2024 – Oct 2024

Project: Balancy Plugin: statistics, dynamic monetization, LiveOps. Role closed when the product line lost its clients.

- Cross-platform plugin in C++20. Roughly halved execution time and memory use.
- Code generator in TypeScript: wrappers for each supported language and platform, integration with Cocos Creator.

Technologies: C++20, TypeScript, Node.js, Cocos Creator, Emscripten, Webpack

Game Programmer (Unreal Engine, GAS), Decartel (Remote)

Jul 2023 – Mar 2024

Project: Malice, MMO RPG with Arkham-style melee combat. Second role, held in parallel with Veom Studio.

- Built the targeting system for melee combat.
- Built the camera system for combat and navigation.
- Gameplay mechanics on UE5's Gameplay Ability System (GAS), stable in multiplayer.
- Client-server game modules; optimized the networking code for responsiveness.

Technologies: C++, Unreal Engine 5, GAS, Git

Senior Software Engineer (Unreal Engine, C++), Veom Studio (Remote)

Jan 2020 – Mar 2024

Project: The Crust (survival-factory game, released on Steam)

- One of the core developers: led design of key modules, ran code reviews, set up the development pipelines.
- Conveyor system: processes 50,000+ resources in under 2 ms, multithreaded.
- 4 FPS to playable (60+), frame time from 250 ms to 16.6 ms. Multithreading plus custom HLSL shaders.
- Save system: save/load times cut by 90%, no loss of data integrity.
- In-game market with dynamic pricing driven by player actions and market conditions.
- Storage systems reworked, ~30% faster.
- Alternative animation system based on vertex colors, ~10× faster in critical scenarios.
- Moon-surface resource generation and mining on customizable noise functions, 2.5M+ resource levels at once.
- Custom rendering pipeline for resource visualization: doubled the frame rate on the Vulkan renderer.

Technologies: C++14, Unreal Engine 4, HLSL, RenderDoc, CMake

ADDITIONAL PROJECTS

- **Game engine side project (2021):** rendering engine in C11 and Vulkan 1.2; batch rendering with a master material and merged meshes to cut draw calls.

EDUCATION & CERTIFICATIONS

- **B.Sc. in Information Technology** - Kaliningrad State Technical University, 2022
- **C++ Programming Professional Certificate** - OpenEDG C++ Institute